

Day 0 - Before You Write Any Code: The Absolute Basics

Day 0 — Before You Write Any Code: The Absolute Basics

This day exists so that every lesson after it can assume you already know what HTML and CSS actually are, how a browser turns them into the page you see on screen, and how to peek "under the hood" of any web page using your browser's built-in developer tools. If you've never built a web page before, read this first — it will make Day 1 dramatically less overwhelming. If any of this is already familiar, skim it and move on.

What actually happens when you visit a web page?

When you open a web page — whether it's a file sitting on your own computer or a real website out on the internet — your **browser** (Chrome, Firefox, Edge, Safari) downloads (or reads, for a local file) a text file written in **HTML**, and then does an enormous amount of work to turn that plain text into the colors, boxes, text, and images you actually see on screen. That whole process is called **rendering**.

Two separate languages work together to make this happen, and today's lesson (and this entire two-week track) is about both of them:

- **HTML** (HyperText Markup Language) describes the *structure and content* of a page — this is a heading, this is a paragraph, this is a picture, this is a link to another page. HTML says **what things are**, not what they look like.
- **CSS** (Cascading Style Sheets) describes the *appearance* of that structure — what color the heading is, how much space surrounds the paragraph, whether the picture is large or small, where things are positioned on the page. CSS says **how things look**.

A useful, genuinely accurate analogy: HTML is like the wooden frame and rooms of a house (this is a bedroom, this is a kitchen, this is the front door) — the structure exists and works correctly even if it's completely unpainted and undecorated. CSS is the paint, furniture, and interior design — it makes the exact same structural rooms look completely different depending on the choices made, without changing what each room fundamentally **is**.

You'll also frequently hear a third language mentioned alongside these two: **JavaScript**, which adds **behavior** — things that happen in response to clicks, typing, or time passing. This particular course only covers HTML and CSS; if you're pairing it with the separate JavaScript/TypeScript track in this same **learning-code** folder, that's where behavior is covered. For these two weeks, you're focused entirely on structure (HTML) and appearance (CSS) — genuinely enough, on its own, to build and style complete, real web pages.

What is a "tag," an "element," and an "attribute"?

These three words get used constantly from Day 1 onward, so let's nail down precisely what each means using one concrete example:

```
<a href="https://example.com">Click here</a>
```

- A **tag** is one specific piece of markup, written between angle brackets: `<a>` is the **opening tag**, and `</a>` (with the forward slash) is the **closing tag**. The name inside the brackets (`a`, here, which stands for "anchor" and is how you make a hyperlink) tells the browser what **kind** of thing this is.
- An **element** is the opening tag, everything between it and its matching closing tag, and the closing tag itself, all together, treated as one unit: the whole thing, `<a href="https://example.com">Click here</a>`, is "an `a` element" (informally, people often just say "a link element" or even just "a link").
- An **attribute** is extra information attached to the **opening** tag, written as `name="value"`: `href="https://example.com"` is an attribute named `href` (short for "hypertext reference"), telling the browser exactly which page this link should go to. An element can have zero, one, or several attributes.

Some elements don't wrap around any content and therefore have no separate closing tag at all — these are called **self-closing** (or "void") elements, and you'll meet a few of these very soon (e.g., `<img>` for an image, `<br>` for a line break).

The minimal shape of an HTML document

Every HTML file follows this same basic skeleton, which you'll type out (or let your editor auto-generate) at the top of essentially every page you ever create:

```
<!DOCTYPE html>
<html>
<head>
  <title>My First Page</title>
</head>
<body>
  <h1>Hello, World!</h1>
  <p>This is a paragraph.</p>
</body>
</html>
```

Reading through this line by line, since you'll want to fully understand every part before Day 1 adds more elements on top of it:

- `<!DOCTYPE html>` isn't really a tag in the same sense as the others — it's a required, one-line declaration telling the browser "interpret everything that follows using the modern, standard rules for HTML." Without it, some older browsers can fall back into inconsistent, legacy rendering behavior — always include it, as the very first line of every HTML file, with nothing before it.
- `<html>...</html>` wraps around the **entire** rest of the document — everything you write lives inside this one element.
- `<head>...</head>` contains information **about** the page that isn't itself displayed as visible content on the page — things like the page's title (shown in the browser tab), links to CSS files (Day 3), and other metadata.
- `<title>My First Page</title>`, inside the `<head>`, sets the text shown in the browser's tab or window title bar — notice this text is never shown **in** the page itself, only in the browser's own interface around it.
- `<body>...</body>` contains everything that actually gets displayed as visible content on the page — headings, paragraphs, images, links, all of it.
- `<h1>Hello, World!</h1>` is a top-level heading (you'll meet the full range of heading levels, `h1` through `h6`, on Day 1).

- `<p>This is a paragraph.</p>` is an ordinary paragraph of text.

Notice the **indentation** (the consistent spacing before `<title>`, `<h1>`, and `<p>`) — unlike Python, HTML's indentation has no effect whatsoever on how the browser interprets the file; it's purely there to help **you**, the human reading the code, visually see which elements are nested inside which other elements. Get in the habit of indenting consistently anyway — your code editor will typically do this for you automatically, and it makes deeply nested pages vastly easier to read later.

Comments — notes to humans that the browser ignores

HTML comments are written as `<!-- like this -->` — anything between those markers is completely ignored by the browser and never shown on the page; it exists purely as a note for whoever reads the source code later (including future-you):

```
<!-- This section is the site's main navigation bar -->
<nav>...</nav>
```

CSS has its own, different comment syntax, which you'll meet properly on Day 3: `/* like this */`.

How to actually open and view an HTML file

Save a file named `hello.html` somewhere on your computer with the skeleton shown above. To view it, you don't need any special program or "running" step the way Python or JavaScript needs an interpreter — you simply open the file directly in your web browser. On Windows, you can typically right-click the file, choose "Open with," and pick your browser — or simply double-click it if your browser is already set as the default program for `.html` files. Once open, you're looking at your browser's **rendering** of that file — the actual visual result of everything you wrote.

Whenever you change the file and want to see the update, you must save the file in your editor, then manually reload the page in your browser (the reload/refresh button, or Ctrl+R / F5) — the browser has no way of knowing you changed the file until you explicitly ask it to reload. This "edit, save, reload, look" cycle is the exact equivalent of Python's "write, run, read output" cycle from that track's Day 0, and you'll repeat it constantly throughout this course.

Browser DevTools — looking under the hood of any web page

Every modern browser has a built-in set of tools, collectively called **DevTools** ("Developer Tools"), for inspecting and even temporarily modifying any page you have open — including pages you didn't write yourself. This is one of the single most useful habits to build starting today, since it lets you directly connect a piece of HTML/CSS you wrote to exactly how the browser actually rendered it.

To open DevTools: right-click anywhere on a page and choose **"Inspect"** (sometimes labeled "Inspect Element"), or press **F12**, or **Ctrl+Shift+I** on Windows. This opens a panel (usually docked to the side or bottom of the browser window) with several tabs — the two you'll use constantly are:

- **Elements** (sometimes called "Inspector" in Firefox) — shows you the actual HTML structure the browser is currently using to render the page, and lets you click on any element to see exactly which CSS rules are

being applied to it, where those rules came from, and even temporarily edit them live to experiment (any changes you make here are temporary and disappear the moment you reload the page — this is a genuinely safe way to experiment without any risk of breaking your actual files).

- **Console** — shows error messages and lets you run small snippets of JavaScript directly against the current page. You won't use this heavily in an HTML/CSS-only course, but it's worth knowing it's there, since browsers will sometimes print warnings here about things like a missing image file or an invalid attribute.

A genuinely useful early habit: whenever a page you're building doesn't look the way you expected, open DevTools, click the **Elements** tab, and click directly on the misbehaving part of the page (there's usually a small cursor/pointer icon in the DevTools toolbar for exactly this — "select an element on the page"). This instantly shows you the exact HTML for that piece, and every single CSS rule currently affecting it — which is, in practice, how real web developers debug almost everything, every single day, rather than guessing.

Exercises

Open `starter.html` in this folder, then open it in your browser to see what it currently looks like. Follow the `<!-- TODO -->` comments inside it, editing the file in your code editor, saving, and reloading your browser after each change to see the result. When you're done, compare against `CHECKLIST.md` and, if you'd like, `solution.html`.